

协程技术调研： 让等待不再占用线程

从并发模型演进到底层实现与实验验证

匡航逸 2412235

并行程序设计调研展示

2026 年 5 月 30 日

核心问题与展示主线

核心判断

协程不是让计算变快，而是让 **等待变便宜**。

- ▶ 大量请求处于 I/O 等待时，线程不应被长期占住。
- ▶ 协程把等待点变成可保存、可恢复的执行状态。
- ▶ 语言、运行时、操作系统必须共同配合。

内容主线

- ① 为什么从线程走到协程
- ② 协程底层如何暂停和恢复
- ③ 常见应用范式如何落地
- ④ 主流语言方案如何取舍
- ⑤ 本机实验与工程边界

问题背景: thread-per-request 的成本

- ▶ 云服务中, 单个请求常常大部分时间在等网络、磁盘、数据库。
- ▶ 若每个请求绑定一个 OS 线程, 等待也会占用线程栈和调度实体。
- ▶ 线程池能限制线程数, 但阻塞任务会把池子占满, 导致排队。



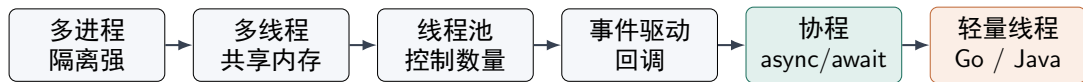
⋮

高并发时: 线程栈、切换、排队成本上升

要解决的问题

不是“一个请求算得更快”, 而是“十万请求同时等待时, 系统还能撑住”。

并发模型的演进路线



问题：创建、栈空间、内核调度、阻塞排队 目标：保留顺序代码体验，同时具备异步 I/O 伸缩性

演进的本质

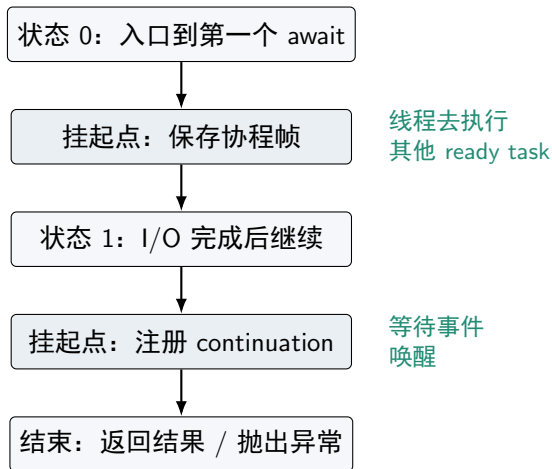
执行单元越来越轻，调度权越来越多地交给语言运行时；但真正的 I/O 事件仍由操作系统提供。

协程是什么：可暂停和恢复的函数

- ▶ 普通函数：沿调用栈执行到返回。
- ▶ 协程：在 `await` / `co_await` / `yield` 处挂起。
- ▶ 挂起时保存执行位置、局部变量、返回值或异常状态。
- ▶ 恢复时从挂起点之后继续执行。

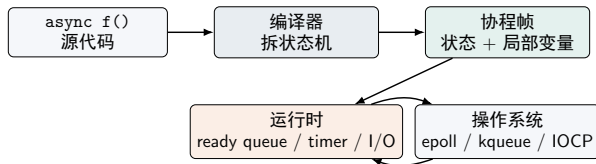
关键区别

协程不是异步 I/O 本身，而是异步控制流的一种表达方式。



底层机制：编译器状态机 + 运行时调度

- ▶ 编译器把异步函数拆成状态机。
- ▶ 跨越挂起点仍存活的数据进入协程帧。
- ▶ 运行时保存 continuation，并在条件满足后恢复。
- ▶ C++20 暴露 promise、coroutine_handle、awaiter 等底层接口。



一句话

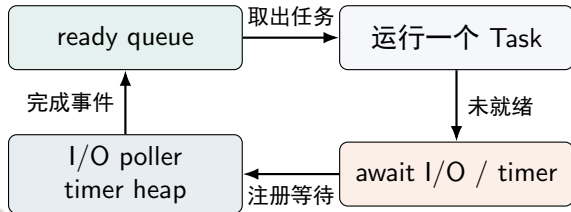
协程“暂停”的本质，是把执行现场从线程栈搬到可保存、可恢复的数据结构中。

事件循环：等待不占线程的关键

- ▶ **ready queue**: 已经可以运行的任务。
- ▶ **timer heap**: 定时器到期后唤醒任务。
- ▶ **I/O poller**: 等待 fd、socket 或系统完成事件。

运行流程

当前协程遇到未就绪 I/O 时挂起，线程立刻切到其他 ready task；I/O 完成后再把它放回队列。

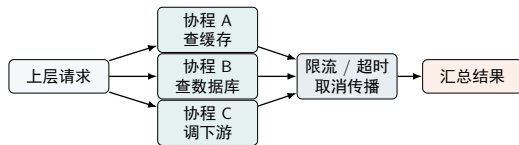


Linux 常见为 `epoll`, macOS/BSD 为 `kqueue`, Windows 为 `IOCP`。

应用范式：把并发写成可控控制流

常见模式

- ▶ **请求级协程**：一个请求一条可挂起控制流。
- ▶ **fan-out / fan-in**：并发访问多个下游，再汇总结果。
- ▶ **生产者-消费者**：异步队列连接爬取、解析、写入等阶段。
- ▶ **受限并发**：信号量、连接池、bounded queue 控制容量。



这类结构的目标是降低总等待时间，同时把资源上限写进程序结构。

关键边界

协程让等待不占线程，但不会扩大下游容量；必须配合超时、取消和限流。

主流语言支持：同一问题的不同取舍

语言	编程接口	调度模型	适合场景与注意点
C++20	co_await, co_yield	标准提供 stackless coroutine 机制；事件循环由库实现	控制力强、开销可控；工程中需要选定 runtime / executor
Go	go f(), channel	Goroutine 由 G-M-P runtime 映射到 OS 线程	服务端体验好；阻塞、GC、共享内存竞争仍要管理
Python	async def, await, asyncio	单事件循环协作式调度 Task	网络 I/O 友好；阻塞函数和 CPU 长任务会卡住事件循环
C#	async/await, Task	编译器状态机 + TaskScheduler / 线程池 continuation	生态完整；注意上下文捕获、Result/Wait 死锁风险
Java 21	Virtual Threads	JVM 用户态线程挂载到 carrier platform thread	保留 thread-per-request 写法；不是 CPU 加速工具，pinning 需关注

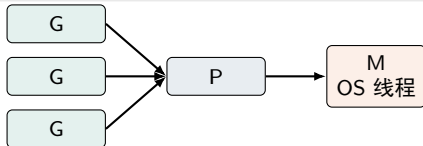
观察

C++ / Python / C# 更像显式 async 控制流；Go / Java 更像由运行时托管的轻量级线程。

Go 与 Java: 轻量级线程路线

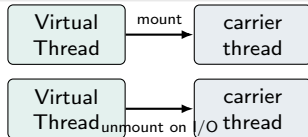
Go Goroutine

- ▶ G: goroutine, 用户态任务。
- ▶ M: machine, OS thread。
- ▶ P: processor, 执行 Go 代码需要的调度资源。



Java Virtual Thread

- ▶ 仍是 `java.lang.Thread` 实例。
- ▶ 可在阻塞 I/O 处 unmount, 释放 carrier thread。
- ▶ 适合保留同步风格的高并发服务。



共同点

让程序员保留接近线程式的写法, 由运行时把大量轻量任务复用到较少 OS 线程上。

实验设计：用 Python 验证协程边界

等待型负载

2000 个任务，每个任务等待 5 ms，验证“等待是否占用线程”。

- ▶ **asyncio**: `asyncio.sleep`, Task 挂起后回到事件循环。
- ▶ **ThreadPool**: `time.sleep`, 等待时占用 worker。
- ▶ **Serial**: 串行等待，作为基线。

CPU 密集负载

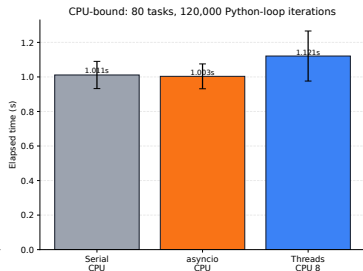
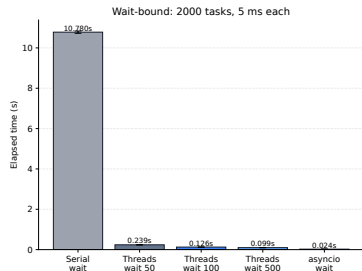
80 个纯 Python 循环任务，每个任务 120000 次整数更新。

- ▶ 对比串行、`asyncio.gather` 和 8 worker 线程池。
- ▶ 用来约束结论：协程不是 CPU 加速工具。

运行环境

- ▶ Python 3.12.7
- ▶ Windows 本机环境
- ▶ 每组重复 5 次，图中展示均值与标准差

实验结果：等待变便宜，计算不变快



等待型方案	平均耗时
Serial	10.780 s
Threads 50	0.239 s
Threads 100	0.126 s
Threads 500	0.099 s
asyncio	0.024 s

CPU 型方案	平均耗时
Serial	1.011 s
asyncio	1.003 s
Threads 8	1.121 s

解释边界

等待型任务中，协程挂起后不占 OS 线程；CPU 任务中，asyncio 与串行同量级，不能替代真正的并行计算。

工程边界：什么时候该用，什么时候不该用

适合协程

- ▶ 高并发网络 I/O：网关、爬虫、代理、消息消费者。
- ▶ 大量任务在等待外部资源。
- ▶ 希望保留接近顺序代码的控制流。

不要误用

- ▶ CPU 密集计算：矩阵、压缩、图像处理、纯计算循环。
- ▶ 需要多核加速时，应考虑线程、进程、SIMD、GPU 或任务并行框架。

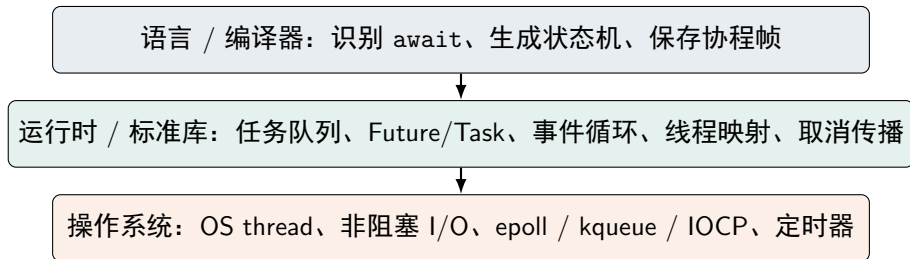
常见坑

- ▶ 在协程里直接调用阻塞 I/O 或 `sleep`。
- ▶ 无限制创建任务，压垮连接池或下游服务。
- ▶ 忽略取消、超时和异常传播。
- ▶ Java virtual thread 的 pinning、C# 的同步等待、Python 的 GIL。

实践准则

限并发、设超时、显式处理取消，用 tracing / profiling 观察运行时行为。

总结：三层协同完成协程机制



最终结论

协程的价值是把“等待”从“占用线程”中分离出来。I/O 密集场景提升吞吐和资源利用率；CPU 密集场景仍要靠真正的并行计算模型。

主要参考资料

- ▶ cppreference: Coroutines (C++20), <https://en.cppreference.com/w/cpp/language/coroutines>
- ▶ Python Documentation: asyncio, Coroutines and Tasks, concurrent.futures, <https://docs.python.org/3/library/asyncio.html>
- ▶ Go Documentation: Effective Go, FAQ, Runtime HACKING, https://go.dev/doc/effective_go
- ▶ Microsoft Learn: Asynchronous programming with async and await, <https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/>
- ▶ OpenJDK: JEP 444 Virtual Threads, <https://openjdk.org/jeps/444>
- ▶ libuv Documentation: Design overview, <https://docs.libuv.org/en/v1.x/design.html>
- ▶ Linux manual pages: epoll(7), pthreads(7), <https://man7.org/linux/man-pages/>